
XSpecMesh: Quality-Preserving Auto-Regressive Mesh Generation Acceleration via Multi-Head Speculative Decoding

Dian Chen^{*1} Yansong Qu^{*1} Xinyang Li¹ Ming Li² Shengchuan Zhang¹

Abstract

Current auto-regressive models can generate high-quality, topologically precise meshes; however, they necessitate thousands—or even tens of thousands—of next-token predictions during inference, resulting in substantial latency. We introduce XSpecMesh, a quality-preserving acceleration method for auto-regressive mesh generation models. XSpecMesh employs a lightweight, multi-head speculative decoding scheme to predict multiple tokens in parallel within a single forward pass, thereby accelerating inference. We further propose a verification and resampling strategy: the backbone model verifies each predicted token and resamples any tokens that do not meet the quality criteria. In addition, we propose a distillation strategy that trains the lightweight decoding heads by distilling from the backbone model, encouraging their prediction distributions to align and improving the success rate of speculative predictions. Extensive experiments demonstrate that our method achieves a $1.7\times$ speedup without sacrificing generation quality. Our code will be released.

1. Introduction

Triangular meshes constitute the foundation of 3D representation and are extensively employed across industries, including virtual reality, gaming, animation, and product design. High-quality meshes exhibiting precise topology are essential for downstream tasks, such as mesh editing, skeletal rigging, texture mapping, and animation. However, constructing meshes with fine-grained topology remains a

labor-intensive endeavor that requires substantial design effort, thus impeding the advancement of 3D content creation. Recent works employ auto-regressive architectures (Siddiqui et al., 2024; Chen et al., 2024a;b; Weng et al., 2025; Zhao et al., 2025a; Liu et al., 2025) for token-based mesh generation, they directly generate mesh vertices and faces while demonstrating the capacity to produce topologically precise meshes. However, the auto-regressive paradigm incurs high inference latency: existing auto-regressive mesh generation models depend on next-token predictions, requiring thousands to tens of thousands of forward passes to produce a single 3D mesh.

We draw inspiration from Speculative Decoding (Leviathan et al., 2023; Chen et al., 2023) in efficient LLM inference, which typically employs a draft model with significantly fewer parameters than the original. The draft model generates candidate tokens, which the original model then verifies—enabling near-draft-model generation speed while preserving the original model’s generation quality. However, draft models must satisfy stringent criteria: their parameter count must be sufficiently constrained to facilitate accelerated inference, and their predictions must closely align with the distribution of the original model. Consequently, deriving such draft models remains a significant challenge (Chen et al., 2023; Leviathan et al., 2023). On the other hand, we note that, unlike auto-regressive language models which frequently employ larger vocabularies to enhance expressiveness (Tao et al., 2024; Huang et al., 2025), existing auto-regressive mesh generation models typically utilize efficient, compressed representations to minimize vocabulary size. Table 1 summarizes these disparities. This discrepancy motivates us to explore a more lightweight decoding design to obtain the probability distribution over the vocabulary.

To this end, we introduce XSpecMesh, a novel framework that accelerates auto-regressive mesh generation models while preserving generation quality. The framework implements multi-head speculative decoding to accelerate inference: multiple lightweight decoding heads simultaneously predict a sequence of subsequent tokens in a single forward pass. These decoding heads leverage cross-attention mechanisms with the generation conditions to enhance prediction accuracy. Furthermore, we introduce a verification and re-

^{*}Equal contribution. Project lead: Yansong Qu. ¹Key Laboratory of Multimedia Trusted Perception and Efficient Computing, Ministry of Education of China, Xiamen University ²Shandong Inspur Database Technology Co., Ltd.. Correspondence to: Shengchuan Zhang <zsc_2016@xmu.edu.cn>.

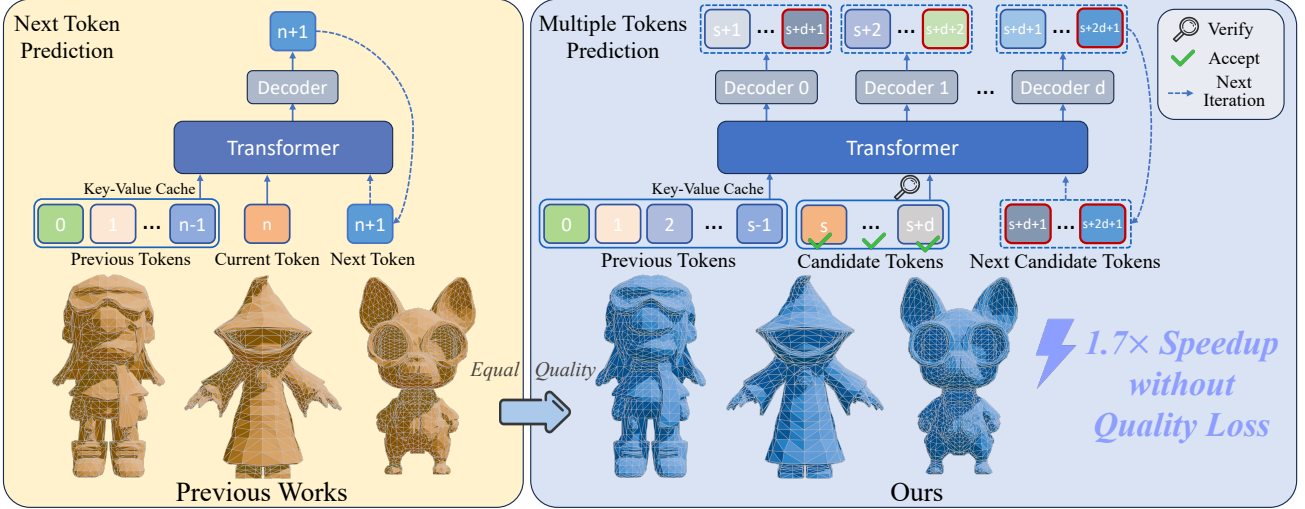


Figure 1. The differences between our framework and previous works. We propose XSpecMesh, a method for accelerating auto-regressive mesh generation models via multi-head speculative decoding, instead of relying on traditional next-token prediction. In a single forward pass, multiple decoding heads predict several tokens, verify the candidate tokens, and resample candidate tokens for the next iteration. Our approach delivers a $1.7\times$ speedup while preserving generation quality.

sampling strategy to evaluate candidate tokens predicted by the decoding heads, resampling those that fail to meet quality criteria, thereby ensuring that output quality remains uncompromised. Finally, we employ backbone distillation training to encourage the decoding heads’ predictive distributions to approximate that of the backbone model, allowing the backbone to accept their predictions. Figure 1 illustrates the differences between our framework and previous works.

We present XSpecMesh, a method for accelerating inference in auto-regressive mesh generation models without sacrificing generation quality. Our contributions can be summarized as follows:

- We propose XSpecMesh, a method to accelerate auto-regressive mesh generation models without compromising generation quality, by employing multiple cross-attention speculative decoding heads for multi-token prediction.
- We develop a verification and resampling strategy that, within a single forward pass, employs the backbone model to verify candidate tokens and resample those that do not meet predefined quality criteria, thereby ensuring uncompromised generation quality.
- We further introduce a distillation strategy to train decoding heads, aligning their prediction distribution with the backbone model to improve the success rate of speculative predictions.
- Extensive experiments demonstrate that our method significantly accelerates inference without sacrificing generation quality, achieving a $1.7\times$ speedup.

2. Related Works

2.1. 3D Mesh Generation

Due to the complexity of direct mesh generation, many 3D synthesis methods utilize intermediate representations—such as voxels (Wu et al., 2016), point clouds (Luo & Hu, 2021; Jun & Nichol, 2023; Qi et al., 2017; Shen et al., 2025c), implicit fields (Chen & Zhang, 2019; Park et al., 2019), or 3DGS (Kerbl et al., 2023; Li et al., 2024; Wang et al., 2024; ?; 2026; Shen et al., 2025b; Qu et al., 2024; Dai et al., 2025; Qu et al., 2025a)—to avoid modeling meshes directly. Representative approaches include optimizing 3D representations within pretrained 2D diffusion models via score-distillation sampling (SDS) (Poole et al., 2022; Wang et al., 2023; Zhu et al., 2023; Li et al., 2023; Lin et al., 2023; Tang et al., 2023; Yi et al., 2024; Li et al., 2024); generating multi-view-consistent images with 2D diffusion models and reconstructing meshes from them (Liu et al., 2023; Shi et al., 2023; Qu et al., 2025b); 3D transformer models (Hong et al., 2023; Tang et al., 2024a; Xu et al., 2024), and the recent 3D latent diffusion models (Zhang et al., 2024; Xiang et al., 2025; Hunyuan3D et al., 2025; Wu et al., 2024; Zhao et al., 2025b) that achieve high-quality shape generation. These approaches typically apply Marching Cubes (Lorensen & Cline, 1998) in post-processing to extract meshes, frequently introducing topological artifacts. In contrast, MeshGPT (Siddiqui et al., 2024), which integrates VQ-VAE (Van Den Oord et al., 2017) with a transformer (Vaswani et al., 2017) for auto-regressive mesh generation, produces high-quality topological meshes; however, it is confined to low-polygon meshes and single-category shapes. A subsequent series of auto-regressive mesh generation

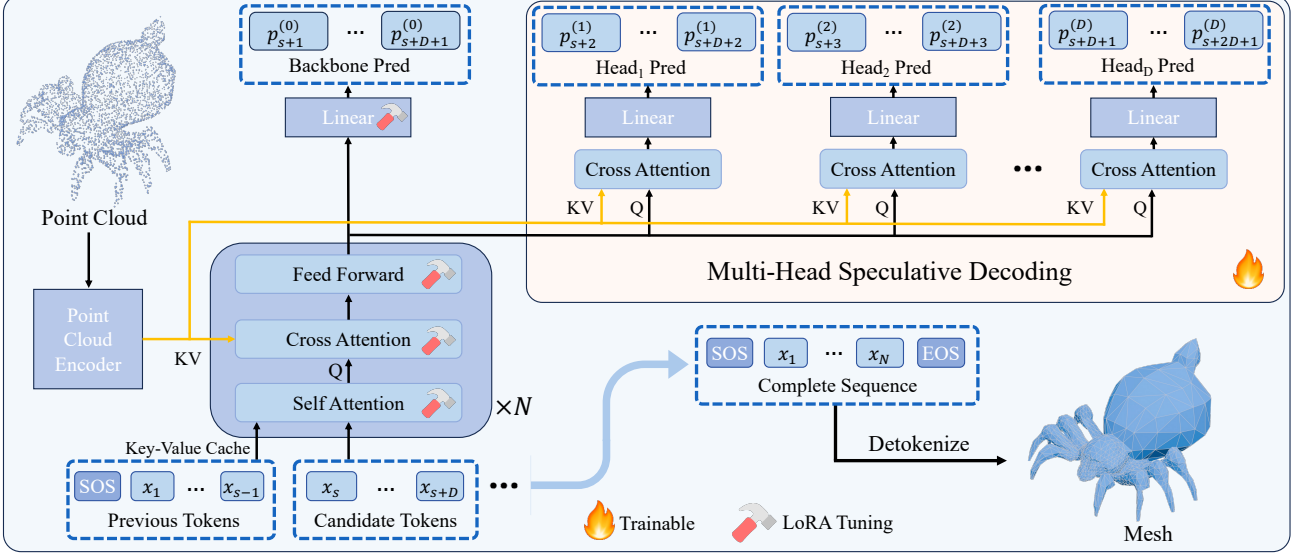


Figure 2. **Overview of our method.** Left: A pretrained transformer-based auto-regressive mesh generation model, fine-tuned with LoRA. Top-right: The transformer’s final hidden layer is decoded by D cross-attention decoding heads, the d -th head predicts the $(d + 1)$ -th next token. Bottom-right: The complete generated token sequence is detokenized to produce the mesh.

methods (Chen et al., 2024b;c; Tang et al., 2024b; Hao et al., 2024; Chen et al., 2024a; Liu et al., 2025), has demonstrated the ability to synthesize topologically precise meshes, BPT (Weng et al., 2025) and DeepMesh (Zhao et al., 2025a) further scale auto-regressive mesh generation to large datasets through efficient tokenization schemes. However, the intrinsic latency of the auto-regressive paradigm hinders its applicability. In this paper, we therefore propose a novel method to accelerate auto-regressive mesh generation while preserving generation quality.

2.2. Acceleration of Auto-Regressive Model

Various strategies have been proposed to accelerate auto-regressive language models: weight pruning methods (Frantar & Alistarh, 2023; Sanh et al., 2020) eliminate redundant parameters to decrease computational load; quantization techniques (Frantar et al., 2022; Xiao et al., 2023) convert models into low-bit representations to cut memory and compute overhead; and sparsity-based approaches (Fedus et al., 2022; Fu et al.) reduce activation computations to improve efficiency. Nonetheless, these methods retain the conventional auto-regressive, token-by-token decoding paradigm. An alternative research direction (Gloeckle et al., 2024; Fan et al., 2025; Cai et al., 2024; Wang et al., 2025) attempts to predict multiple tokens in a single forward pass to reduce iterative decoding steps. The Speculative Decoding approaches (Chen et al., 2023; Sun et al., 2023; Leviathan et al., 2023) employ a draft model to generate tokens rapidly, then verify them with the original model to preserve generation quality. Certain efforts target acceleration of auto-regressive image synthesis: SJD (Teng et al., 2024) inte-

grates Speculative Decoding with Jacobi decoding, whereas ZipAR (He et al., 2024) exploits local sparsity for parallel token generation. To date, these acceleration studies have focused predominantly on language and image generation domains, with auto-regressive mesh generation remaining insufficiently explored.

3. Preliminary

3.1. Auto-Regressive Mesh Generation

An auto-regressive mesh generation framework comprises three fundamental components: a discrete mesh serialization method (Chen et al., 2024b; Weng et al., 2025) that converts vertices and faces into a token sequence; a transformer-based auto-regressive generator that, conditioned on input prompts, sequentially predicts each subsequent token to generate the token sequence; a deserialization method that reconstructs the 3D mesh vertices and faces from the generated sequence.

Auto-regressive models employ causal masking during training, so that, for a given sequence $x_{0:n}$, the model can perform, in a single forward pass, simultaneous computations of the predictive distributions for positions $1, 2, \dots, n + 1$:

$$p_1(x|x_0), p_2(x|x_{0:1}), \dots, p_{n+1}(x|x_{0:n}). \quad (1)$$

For each position i , with corresponding target label y_i , the model is trained by minimizing the cross-entropy loss:

$$\mathcal{L} = \sum_i -\log p_i(y_i). \quad (2)$$

This property also means that, at inference time, by evaluating $p_{i+1}(x|x_{0:i})$, one can determine whether a candidate token x_{i+1} aligns with the model’s learned distribution. Our method leverages this property to accelerate generation without compromising quality.

4. Method

Our method aims to accelerate auto-regressive mesh generation models without compromising generation quality. We propose multi-head speculative decoding, in which multiple lightweight cross-attention decoding heads concurrently predict subsequent tokens, thereby accelerating the sequence generation process (Sec 4.1). Since these decoding heads’ predictions may be imprecise, we employ the backbone model’s robust prior to verify outputs—rejecting and re-sampling at the first invalid token—to guarantee generation quality (Sec 4.2). To enhance acceptance of decoding heads’ proposals, we distill backbone knowledge into these heads during training, aligning their output distributions with the backbone’s (Sec 4.3). Figure 2 provides an overview of our method.

4.1. Multi-Head Speculative Decoding

Auto-regressive models exhibit excellent generation quality, however, their inference relies on sequential, token-by-token generation, leading to high latency. To alleviate this bottleneck, we introduce multi-head speculative decoding. In auto-regressive mesh generation models, the vocabulary size is considerably smaller than that of LLMs (Table 1), resulting in a relatively simple decoding process. Therefore, we propose a more efficient approach that employs multiple lightweight decoding heads to process the transformer’s final hidden layer and predict subsequent tokens.

Specifically, the backbone model comprises N transformer blocks, each containing: a self-attention layer, a cross-attention layer for injecting the generation condition c , and a feed-forward network. Let s denote the current sequence position, and assume tokens x_0 through x_{s-1} are stored in the key-value cache. Denote the layer-0 hidden state as $h_s^0 = x_s$. Then, for $l = 0, 1, \dots, N-1$, the $(l+1)$ -th hidden state is computed as $h_s^{l+1} = \text{block}^l(h_s^l, c)$. Define the final hidden state as $h_s = h_s^N$. The backbone model subsequently decodes h_s through a linear layer $W^{(0)}$ to yield the probability distribution for the next token at position $s+1$:

$$p_{s+1}^{(0)} = \text{softmax}(W^{(0)} \cdot h_s). \quad (3)$$

Given the generation condition c , we employ multiple cross-attention decoding heads to decode h_s , with the d -th decod-

Method	BPT	DeepMesh	LLaMa 3	Qwen3
Vocab Size	5120	4736	128K	152K

Table 1. The difference in vocabulary size between auto-regressive mesh generation models and language models. Language models (Grattafiori et al., 2024; Yang et al., 2025) tend to use larger vocabularies to enhance expressiveness, whereas auto-regressive mesh generation models favor efficient compressed representations to reduce vocabulary size.

ing head predicting the token at position $s+d+1$:

$$p_{s+d+1}^{(d)} = \text{softmax}(W^{(d)} \cdot \text{CrossAttn}^{(d)}(h_s, c)). \quad (4)$$

Compared to decoding via an MLP, using a cross-attention mechanism allows the decoding heads to better align with the input conditional features, thereby improving the accuracy of subsequent-token predictions. Finally, we sample from probability distributions $p_{s+1}^{(0)}, p_{s+2}^{(1)}, \dots, p_{s+D+1}^{(D)}$ to generate the tokens $x_{s+1}, x_{s+2}, \dots, x_{s+D+1}$.

4.2. Verification and Resampling

After generating the next $D+1$ tokens from position s via the backbone model and D decoding heads, the straightforward approach is to append these tokens to the existing sequence and resume prediction at position $s+D+1$. However, due to potential inaccuracies of the decoding heads, this strategy can drastically degrade the generated sequence’s quality. We therefore propose a verification strategy that leverages the backbone model to simultaneously verify and resample tokens in a single forward pass.

Specifically, we leverage the backbone model’s prior judgment to determine whether to accept tokens predicted by the decoding heads. Let s denote the current accepted sequence position. In a single forward pass, the backbone model employs causal masking on the sequence $x_{s:s+D}$ to obtain $p_{s+1:s+D+1}^{(0)}$, and based on this probability distribution, partially accepts a prefix $x_{s:s^*-1}$. Subsequently, with the backbone model and D decoding heads, we resample tokens at positions s^* to s^*+D . We apply a probability-threshold-based criterion: a token x_i is accepted if $p_i^{(0)}(x_i) > \delta$. Figure 3 provides a detailed illustration of this process.

By verifying with the backbone model and sampling with multiple decoding heads, we reduce the number of forward passes through the backbone model while preserving generation quality, thus speeding up the overall generation process. Algorithm 1 presents a detailed description of the multi-head speculative decoding procedure.

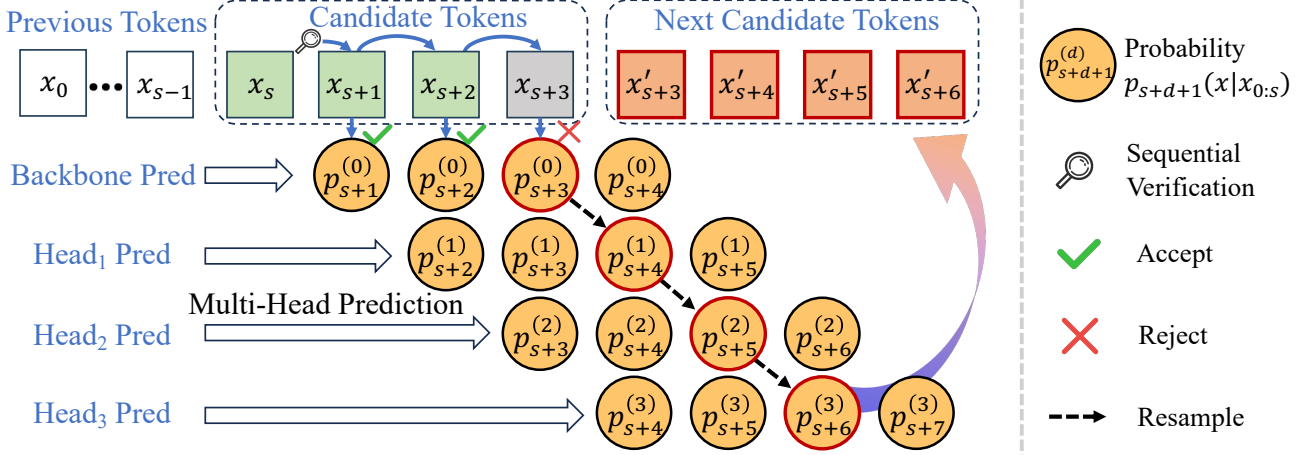


Figure 3. **Verification and resampling.** The figure uses $D = 3$ as an example to illustrate the process. Each candidate token sampled in a forward pass must be verified by the backbone model: if $p_i^{(0)}(x_i) > \delta$, token x_i is accepted and verification proceeds to the next token, until the first token $x_{i'}$ that fails the verification condition. Then resample a token at position i' , forming the candidate tokens for the next iteration.

4.3. Backbone Distillation Training

Analogous to Speculative Decoding, in which the draft model’s output distribution must closely match that of the original model, our framework requires the decoding heads’ output distributions to align with the backbone model’s distribution to ensure acceptance of their predictions. To this end, we distill the backbone model to train decoding heads. We sample point clouds from the dataset and employ the backbone model to generate sequences, which serves as the ground truth labels $y_{0:n}$ for decoding heads training. We train the d -th decoding head using the cross-entropy loss:

$$\mathcal{L}_d = \sum_s -\log p_{s+d+1}^{(d)}(y_{s+d+1}). \quad (5)$$

With increasing d , the accuracy of the d -th decoding head declines, potentially causing gradient instability. To mitigate this issue, we introduce a weighting function $w(d)$, which decreases as d increases. Accordingly, the overall loss for the D decoding heads is formulated as follows:

$$\mathcal{L}_{\text{mhd}} = \sum_{d=1}^D w(d) \cdot \mathcal{L}_d. \quad (6)$$

Following decoding heads training, they are deployed for inference acceleration. Empirical evaluation, however, indicates that the speed-up benefits are modest. This limitation arises because the backbone model is optimized under a next-token prediction paradigm, making direct decoding of subsequent tokens from the hidden state h_s infeasible. To mitigate this issue, we fine-tune the backbone model’s linear layer via LoRA (Hu et al., 2022), enabling the decoding heads to more effectively derive multiple subsequent

token predictions from h_s . Training proceeds in two stages. In the first stage, we train only the decoding heads while freezing the backbone model to prevent unstable gradients from the decoding heads in the early training stage from affecting the backbone model. In the second stage, we jointly train both the decoding heads and LoRA. Furthermore, we integrate the backbone model’s prediction loss $\mathcal{L}_{\text{backbone}} = \sum_s -\log p_{s+1}^{(0)}(y_{s+1})$ into the overall objective with a substantial weighting factor λ , ensuring gradients from the decoding heads do not diverge the backbone distribution from its original form. The loss function for the second stage is formulated as follows:

$$\mathcal{L}_{\text{total}} = \lambda \mathcal{L}_{\text{backbone}} + \mathcal{L}_{\text{mhd}}. \quad (7)$$

Although during training LoRA introduces two low-rank matrices A and B to each original linear layer weight matrix W , at inference time these LoRA weights can be merged with the original weights W_{origin} via a simple preprocessing step to form the merged weight $W_{\text{merge}} = W_{\text{origin}} + AB$. Therefore, introducing LoRA incurs no additional computational overhead. Upon fine-tuning the backbone model with LoRA, the decoding heads are able to accurately predict subsequent tokens, significantly increasing the decoding speed.

5. Experiments

5.1. Experiment Settings

Serialization strategy. We adopt the same serialization strategy as BPT (Weng et al., 2025), which is an irregular compressed mesh serialization method. Specifically, BPT hierarchically encodes mesh vertices using block indices and offset indices to balance vocabulary size and sequence

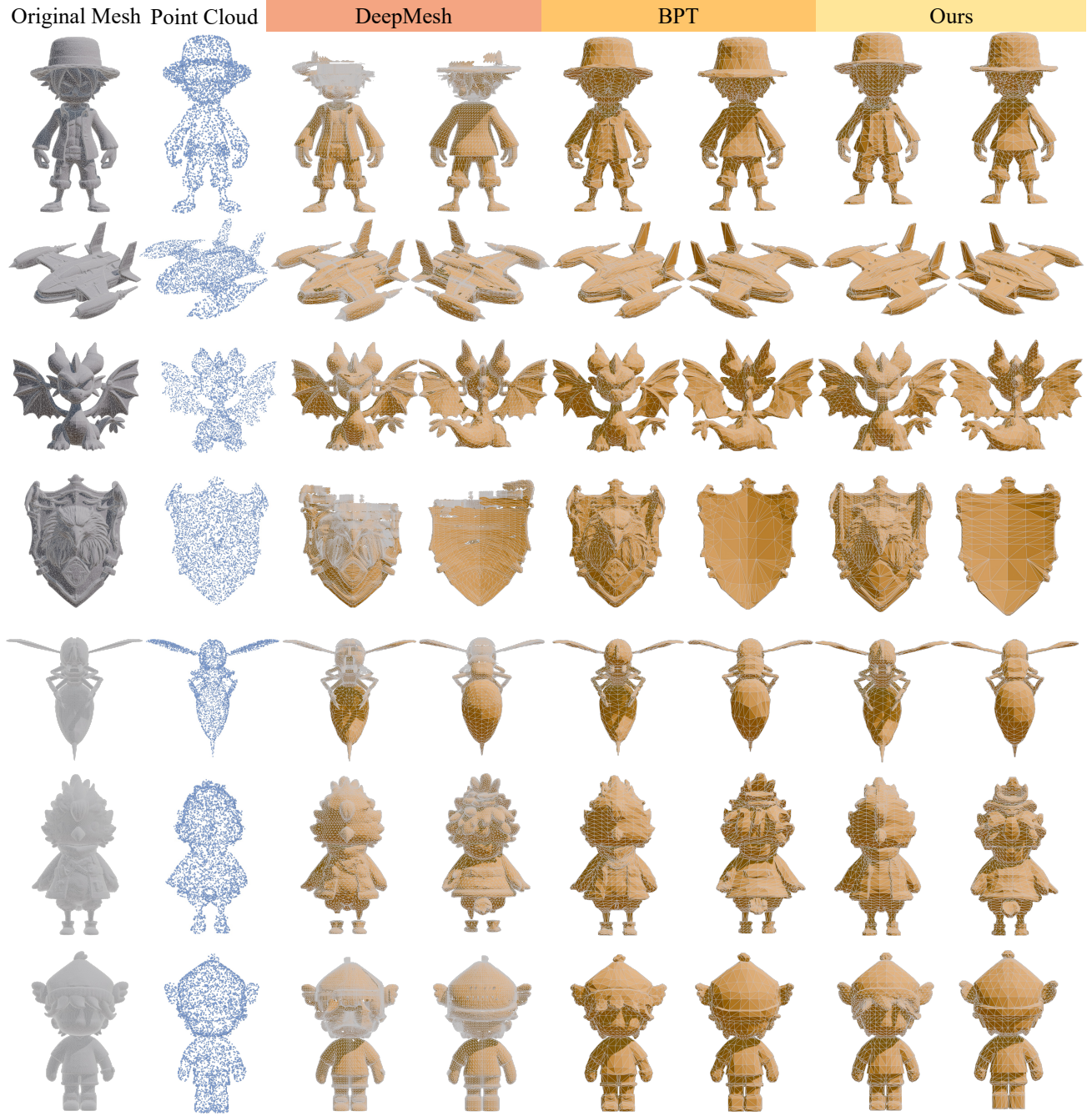


Figure 4. Comparison of our method with the base model BPT and another mesh generation model DeepMesh. Our acceleration method, built upon BPT, substantially accelerates generation while preserving BPT’s shape and topological fidelity.

length, and further applies patchify operations to compress the substantial redundant repetition of vertices in the sequence. Our method can effectively accelerate inference under such irregular compressed sequences.

Implementation Details. We adopt BPT as our base model: an auto-regressive mesh generation model pretrained on a large-scale, high-quality dataset. We train on a subset of

Objaverse (Deitke et al., 2023) containing approximately 10K shapes. In the first stage, we train only the decoding heads, setting the loss weight for the d -th decoding head to $w(d) = 0.8^d$. In the second stage, we jointly train the LoRA adapters and the decoding heads; to prevent the backbone model’s distribution from drifting, we assign a relatively large weight $\lambda = 50$ to the backbone loss. See the Appendix for more details.

Algorithm 1 Multi-Head Speculative Decoding

Input: Condition c , Backbone Model $\mathcal{M}$, Multi-Head Speculative Decoder $\{\mathcal{H}_i\}_{i=1}^D$

Output: Mesh Sequence $x_{0:i_{\text{EOS}}}$

```

1: Let  $x_0 \leftarrow \text{SOS}$ ,  $x_{1:D} \sim U(0, V)$ ,  $s \leftarrow 0$ .
2: while  $s < L_{\text{max}}$  and  $x_{0:s} \neq \text{EOS}$  do
3:    $p_{s+1:s+D+1}^{(0)}$ ,  $h_{s:s+D} \leftarrow \mathcal{M}(x_{s:s+D}, c)$ 
   {forward with causal mask}
4:   for  $i = 1$  to  $D$  do
5:      $p_{s+1+i:s+D+1+i}^{(i)} \leftarrow \mathcal{H}_i(h_{s:s+D}, c)$ 
6:   end for
7:    $s^* \leftarrow s + 1$ 
8:   while  $s^* < s + D + 1$  and  $p_{s^*}^{(0)}(x_{s^*}) > \delta$  do
9:      $s^* \leftarrow s^* + 1$  {verify and accept}
10:  end while
11:   $x_{s^*:s^*+D} \leftarrow \text{sample}(p_{s^*}^{(0)}, p_{s^*+1}^{(1)}, \dots, p_{s^*+D}^{(D)})$ 
   {resample from the first rejected position  $s^*$ }
12:   $s \leftarrow s^*$ 
13: end while
14: return  $x_{0:i_{\text{EOS}}}$ 
    
```

Baselines. We compare our method against the base model BPT and another state-of-the-art auto-regressive mesh generation model, DeepMesh (Zhao et al., 2025a). Since DeepMesh has only released a 0.5B-parameter configuration, we use this version for evaluation.

Metrics. We follow the evaluation procedure of previous work (Weng et al., 2025; Zhao et al., 2025a; Liu et al., 2025), and generate 200 test meshes via the generation model (Xiang et al., 2025; Zhao et al., 2025b) (see the Appendix for more details). We uniformly sample 1,024 points from the surfaces of ground-truth and generated meshes, computing Chamfer Distance (CD) and Hausdorff Distance (HD) as objective quality metrics. Additionally, a user study (US) is conducted to capture subjective assessments. For speedup evaluation, we follow the methodology of previous work (Fu et al., 2024; Xia et al., 2022; Chen et al., 2023) and define the Step Compression Ratio as: $\text{SCR} = \frac{\text{number of generated and accepted tokens}}{\text{number of decoding steps}}$, where a decoding step denotes the process of verifying and decoding multiple tokens in a single forward pass. Since we introduced additional decoding heads, we measured the latency of a single decoding step (Step Latency) on an RTX 3090. Finally, we computed the actual speedup ratio (Speedup) based on SCR and Step Latency.

5.2. Qualitative Results

We perform a qualitative comparison of our method against established baselines, presenting several challenging examples in Figure 4. Although DeepMesh can generate higher-resolution meshes, its truncated-window training induces

Method	CD ↓	HD ↓	US ↑	Avg. Lat. ↓
DeepMesh*	0.1323	0.2648	27%	979.6s
BPT	0.1165	0.2223	37%	257.6s
Ours	0.1168	0.2261	36%	151.4s

Table 2. Quantitative comparison with other methods. Our approach achieves generation quality comparable to the base model BPT while delivering significantly faster generation speed than BPT. Avg. Lat. denotes the average latency to generate the complete mesh sequence (measured on the RTX 3090). DeepMesh* was tested using its 0.5B version.

context loss, resulting in fragmented meshes. In contrast, BPT yields more consistent generation results, while our approach achieves shape and topological fidelity comparable to BPT.

5.3. Quantitative Results

Table 2 summarizes the results of our quantitative comparison. DeepMesh is capable of generating high-resolution meshes, which has earned it a certain level of popularity in user study. However, owing to its propensity to produce fragmented and incomplete meshes, DeepMesh exhibits higher CD and HD values. By contrast, the results generated by BPT demonstrate greater consistency. Since our method produces results highly similar to BPT, the corresponding CD and HD metrics are comparable. Moreover, in the user study where methods were anonymized, participants were unable to differentiate between our method’s outputs and those of BPT, yielding comparable survey scores. Overall, our method matches the baseline BPT in generation quality while significantly reducing complete mesh sequence generation latency.

5.4. Ablation Study

Decoding head architectures and training strategies. We first compared the quality of the generated shapes and the achieved speed-up under different decoding head architectures and training strategies: A. Baseline model: BPT; B. MLP decoding heads, training only the first-stage decoding heads; C. MLP decoding heads, first training the first-stage decoding heads, then jointly training LoRA adapters and decoding heads in a second stage; D. Cross-attention decoding heads, training only the first-stage decoding heads; E. Cross-attention decoding heads, first training the first-stage decoding heads, then jointly training LoRA adapters and decoding heads in a second stage.

Table 3 presents the evaluation results for different configurations. Compared to the MLP decoding heads, the cross-attention decoding heads, despite incurring higher step latency, more effectively integrate conditional information into the generation process, thereby yielding more accurate

Configuration	CD ↓	HD ↓	SCR ↑	Step Latency ↓	Speedup ↑
A BPT	0.1165	0.2223	1.000	40.51ms	1.00×
B w. MLP Decoder	0.1195	0.2241	1.181	44.89ms	1.07×
C w. MLP Decoder & LoRA	0.1267	0.2485	1.909	44.92ms	1.65×
D w. CA Decoder	0.1167	0.2229	1.334	47.81ms	1.13×
E w. CA Decoder & LoRA (Ours)	0.1168	0.2261	2.021	47.83ms	1.71×

Table 3. **Ablation across different configurations.** We compare MLP decoding heads versus Cross-Attention (CA) decoding heads, and evaluate the effect of two-stage LoRA joint training with the decoding heads. The Cross-Attention decoding heads incorporate generation conditions, achieving excellent performance in both generation quality and speedup.

predictions of subsequent tokens and consequently improving the SCR. After two-stage joint training with LoRA, the MLP decoding heads also achieve a comparably high speedup; however, their generation quality deteriorates to some extent. This degradation stems mainly from (1) Joint training with LoRA aligns the prediction distributions of the decoding heads with those of the backbone model, thereby increasing the backbone’s propensity to accept the decoding head’s outputs, and (2) the MLP decoding heads’ predictions, lacking injected conditional information, produce some inaccuracies that the backbone model still accepts, thereby compromising overall quality. In contrast, integrating cross-attention decoding heads with LoRA joint training better aligns multi-token predictions with generation condition, resulting in superior performance in both generation quality and speedup ratio.

Number of decoding heads. Increasing the number of decoding heads raises SCR but also increases step latency. As shown in Figure 5(a), we present SCR and step latency for various numbers of decoding heads and subsequently compute speedup. At $D = 4$, speedup peaks at $1.71\times$.

Verification criterion. We use a threshold δ as the acceptance condition: a token x_i is accepted if $p_i^{(0)}(x_i) > \delta$. As the hyperparameter δ increases, the criterion becomes stricter, leading to lower speedup but improved generation quality. Figure 5(b) illustrates the impact of varying δ on speedup, CD, and HD. At $\delta = 0.5$, our method achieves an optimal trade-off between speedup and generation quality, delivering substantial acceleration while preserving quality comparable to the baseline model. Furthermore, we compare two acceptance criteria—Probability-Threshold Acceptance and Top- K_a Acceptance (a token x_i is accepted if x_i is among the top- K_a tokens of $p_i^{(0)}$)—and present the results in Figure 6. For top- K_a acceptance, a long-tail effect arises: certain candidate tokens within the top- K_a may exhibit exceedingly low probabilities yet be accepted, severely degrading generation quality. Only for $K_a = 1$ does the model generate a reasonable shape. By contrast, probability-threshold acceptance demonstrates greater stability, yielding satisfactory results for thresholds between 0.1 and 0.5.

Sampling strategies. We compare two sampling strategies: Independent Sampling and Top- K_s Probability-Tree Sampling, see the Appendix for details.

5.5. Watertightness and Manifoldness

Watertightness and manifoldness are important metrics for assessing whether generated meshes can be reliably used in downstream applications. To further verify that our method improves generation efficiency without sacrificing geometric validity, we conduct additional evaluations on watertightness and manifoldness for both existing autoregressive mesh generation methods and our proposed method. These evaluations complement standard generation-quality metrics by focusing on the topological and structural validity of the generated meshes, rather than only measuring their visual or distributional quality. Specifically, watertightness reflects whether a generated mesh forms a closed surface without holes or boundary gaps, while manifoldness indicates whether the local connectivity around vertices and edges satisfies valid surface constraints. Detailed quantitative results, evaluation protocols, and further analysis are provided in the Appendix. It is worth noting that autoregressive mesh generation is still at an early stage, and most existing methods primarily focus on producing plausible geometry rather than explicitly enforcing watertightness or manifoldness during generation. As a result, current methods still struggle to consistently produce outputs that are both watertight and manifold, suggesting that improving topological validity remains an important direction for future work.

6. Discussion

Limitation. Although our method accelerates the base model’s generation speed without sacrificing output quality, it still relies on the base model as the backbone to verify candidate tokens and preserve generation quality. As a result, the overall performance of our approach remains tied to the capability and efficiency of the underlying base model. Nevertheless, this design allows our method to inherit the strengths of the backbone while providing consistent accel-

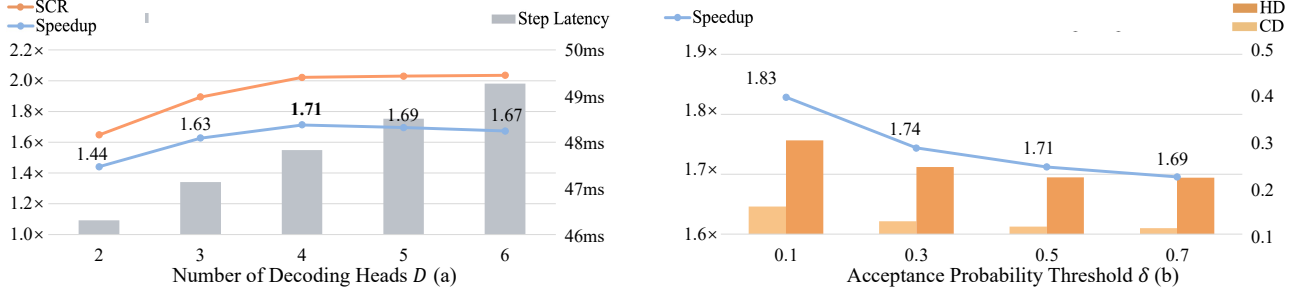


Figure 5. Left: ablation of the number of decoding heads D ; speedup peaks at $D = 4$. Right: ablation of the acceptance probability threshold δ ; at $\delta = 0.5$, generation quality matches the base model while speedup exceeds $1.7\times$.

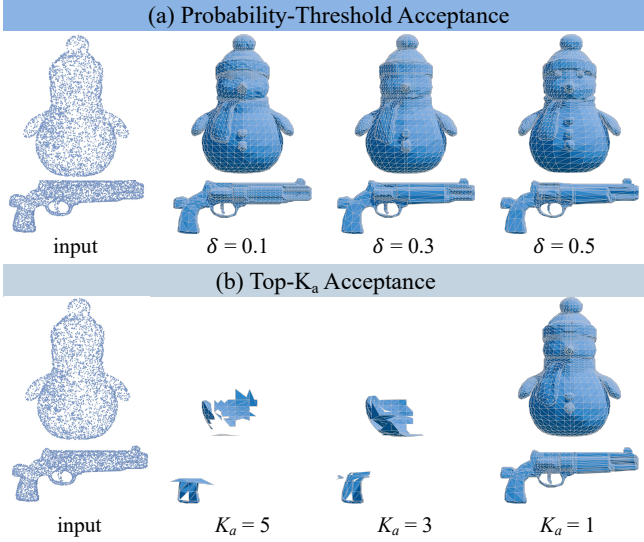


Figure 6. **Comparison of Probability-Threshold and Top- K_a Acceptance.** Probability-threshold acceptance is more stable, generating reasonable shapes across thresholds.

eration in practice.

Acceleration Bottlenecks. Although our method consistently accelerates autoregressive mesh generation across different settings, the achieved speedup is still lower than the $2\times$ – $3\times$ gains commonly reported for speculative decoding in large language models. We attribute this gap to several characteristics specific to autoregressive mesh generation. First, existing autoregressive mesh generation models are typically much smaller than modern LLMs. As a result, they contain less parameter redundancy, which limits the potential benefit of using a lightweight draft model to approximate the target model. Second, mesh token sequences often exhibit stronger local and structural dependencies than natural language sequences. A single geometric element, such as a vertex coordinate, face index, or local connectivity pattern, may be serialized into multiple tightly coupled tokens, so an error in one token can easily affect the validity of subsequent tokens. This makes accurate multi-token

drafting and acceptance more challenging. Third, recent mesh tokenization and serialization strategies are usually designed to be compact, reducing unnecessary sequence length and repeated patterns. While such compact representations improve generation efficiency in the standard autoregressive setting, they also leave less sequence-level redundancy for speculative decoding to exploit. Consequently, the attainable acceleration is naturally more limited than in LLM generation. Nevertheless, our method still provides consistent and practical speedups without degrading mesh generation quality, suggesting that speculative acceleration remains a promising direction for efficient autoregressive mesh generation.

7. Conclusion

We propose XSpecMesh, a speculative decoding framework for accelerating auto-regressive mesh generation models through multiple cross-attention decoding heads for multi-token prediction. By leveraging multi-head speculative decoding together with a verification and resampling strategy, our method improves inference efficiency while preserving generation quality. Extensive experiments show that XSpecMesh achieves up to a $1.7\times$ speedup over the base model without compromising output quality.

Acknowledgements

This work was supported by the National Science and Technology Major Project (No. 2025YFE0113500), the National Science Fund for Distinguished Young Scholars (No. 62525605), and the National Natural Science Foundation of China (No. U25B2066, No. U22B2051, and No. 62272401).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Chen, S., Chen, X., Pang, A., Zeng, X., Cheng, W., Fu, Y., Yin, F., Wang, B., Yu, J., Yu, G., et al. Meshxl: Neural coordinate field for generative 3d foundation models. *Advances in Neural Information Processing Systems*, 37: 97141–97166, 2024a.
- Chen, Y., He, T., Huang, D., Ye, W., Chen, S., Tang, J., Chen, X., Cai, Z., Yang, L., Yu, G., et al. Meshanything: Artist-created mesh generation with autoregressive transformers. *arXiv preprint arXiv:2406.10163*, 2024b.
- Chen, Y., Wang, Y., Luo, Y., Wang, Z., Chen, Z., Zhu, J., Zhang, C., and Lin, G. Meshanything v2: Artist-created mesh generation with adjacent mesh tokenization. *arXiv preprint arXiv:2408.02555*, 2024c.
- Chen, Z. and Zhang, H. Learning implicit fields for generative shape modeling. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5939–5948, 2019.
- Dai, S., Qu, Y., Li, Z., Li, X., Zhang, S., and Cao, L. Training-free hierarchical scene understanding for gaussian splatting with superpoint graphs. *arXiv preprint arXiv:2504.13153*, 2025.
- Deitke, M., Schwenk, D., Salvador, J., Weihs, L., Michel, O., VanderBilt, E., Schmidt, L., Ehsani, K., Kembhavi, A., and Farhadi, A. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 13142–13153, 2023.
- Fan, X., Sun, Z., Gao, Y., Xiong, J., Yan, H., Cao, Y., Sun, J., Li, S., Zhang, Z., Xi, Z., et al. Speech-language models with decoupled tokenizers and multi-token prediction. *arXiv preprint arXiv:2506.12537*, 2025.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Frantar, E. and Alistarh, D. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*, pp. 10323–10337. PMLR, 2023.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Fu, T., Huang, H., Ning, X., Zhang, G., Chen, B., Wu, T., Wang, H., Huang, Z., Li, S., Yan, S., et al. Moa: Mixture of sparse attention for automatic large language model compression. In *ICLR 2025 Workshop on Foundation Models in the Wild*.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of llm inference using lookahead decoding. *arXiv preprint arXiv:2402.02057*, 2024.
- Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Hao, Z., Romero, D. W., Lin, T.-Y., and Liu, M.-Y. Meshtron: High-fidelity, artist-like 3d mesh generation at scale. *arXiv preprint arXiv:2412.09548*, 2024.
- He, Y., Chen, F., He, Y., He, S., Zhou, H., Zhang, K., and Zhuang, B. Zipar: Parallel autoregressive image generation through spatial locality. In *Forty-second International Conference on Machine Learning*, 2024.
- Hong, Y., Zhang, K., Gu, J., Bi, S., Zhou, Y., Liu, D., Liu, F., Sunkavalli, K., Bui, T., and Tan, H. Lrm: Large reconstruction model for single image to 3d. *arXiv preprint arXiv:2311.04400*, 2023.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Huang, H., Zhu, D., Wu, B., Zeng, Y., Wang, Y., Min, Q., and Zhou, X. Over-tokenized transformer: Vocabulary is generally worth scaling. *arXiv preprint arXiv:2501.16975*, 2025.
- Hunyuan3D, T., Yang, S., Yang, M., Feng, Y., Huang, X., Zhang, S., He, Z., Luo, D., Liu, H., Zhao, Y., et al. Hunyuan3d 2.1: From images to high-fidelity 3d assets with production-ready pbr material. *arXiv preprint arXiv:2506.15442*, 2025.
- Jun, H. and Nichol, A. Shap-e: Generating conditional 3d implicit functions. *arXiv preprint arXiv:2305.02463*, 2023.

- Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):139–1, 2023.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.
- Li, W., Chen, R., Chen, X., and Tan, P. Sweetdreamer: Aligning geometric priors in 2d diffusion for consistent text-to-3d. *arXiv preprint arXiv:2310.02596*, 2023.
- Li, X., Lai, Z., Xu, L., Qu, Y., Cao, L., Zhang, S., Dai, B., and Ji, R. Director3d: Real-world camera trajectory and 3d scene generation from text. *Advances in neural information processing systems*, 37:75125–75151, 2024.
- Lin, C.-H., Gao, J., Tang, L., Takikawa, T., Zeng, X., Huang, X., Kreis, K., Fidler, S., Liu, M.-Y., and Lin, T.-Y. Magic3d: High-resolution text-to-3d content creation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 300–309, 2023.
- Lionar, S., Liang, J., and Lee, G. H. Treemeshgpt: Artistic mesh generation with autoregressive tree sequencing. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 26608–26617, 2025.
- Liu, J., Xu, J., Guo, S., Li, J., Guo, J., Yu, J., Weng, H., Lei, B., Yang, X., Chen, Z., et al. Mesh-rft: Enhancing mesh generation via fine-grained reinforcement fine-tuning. *arXiv preprint arXiv:2505.16761*, 2025.
- Liu, R., Wu, R., Van Hoorick, B., Tokmakov, P., Zakharov, S., and Vondrick, C. Zero-1-to-3: Zero-shot one image to 3d object. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 9298–9309, 2023.
- Lorensen, W. E. and Cline, H. E. Marching cubes: A high resolution 3d surface construction algorithm. In *Seminal graphics: pioneering efforts that shaped the field*, pp. 347–353. 1998.
- Luo, S. and Hu, W. Diffusion probabilistic models for 3d point cloud generation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 2837–2845, 2021.
- Park, J. J., Florence, P., Straub, J., Newcombe, R., and Lovegrove, S. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 165–174, 2019.
- Poole, B., Jain, A., Barron, J. T., and Mildenhall, B. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022.
- Qi, C. R., Su, H., Mo, K., and Guibas, L. J. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 652–660, 2017.
- Qu, Y., Dai, S., Li, X., Lin, J., Cao, L., Zhang, S., and Ji, R. Goi: Find 3d gaussians of interest with an optimizable open-vocabulary semantic-space hyperplane. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 5328–5337, 2024.
- Qu, Y., Chen, D., Li, X., Li, X., Zhang, S., Cao, L., and Ji, R. Drag your gaussian: Effective drag-based editing with score distillation for 3d gaussian splatting. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, pp. 1–12, 2025a.
- Qu, Y., Dai, S., Li, X., Wang, Y., Shen, Y., Cao, L., and Ji, R. Deocc-1-to-3: 3d de-occlusion from a single image via self-supervised multi-view diffusion. *arXiv preprint arXiv:2506.21544*, 2025b.
- Sanh, V., Wolf, T., and Rush, A. Movement pruning: Adaptive sparsity by fine-tuning. *Advances in neural information processing systems*, 33:20378–20389, 2020.
- Shen, T., Zhang, Y., Tang, C., Ping, C., Zhao, Z., Wan, L., Wang, Y., Wang, R., and He, S. Flashmesh: Faster and better autoregressive mesh synthesis via structured speculation. *arXiv preprint arXiv:2511.15618*, 2025a.
- Shen, Y., Zhang, Z., Li, X., Qu, Y., Lin, Y., Zhang, S., and Cao, L. Evolving high-quality rendering and reconstruction in a unified framework with contribution-adaptive regularization. *arXiv preprint arXiv:2503.00881*, 2025b.
- Shen, Y., Zhang, Z., Qu, Y., Zheng, X., Ji, J., Zhang, S., and Cao, L. Fastvggt: Training-free acceleration of visual geometry transformer. *arXiv preprint arXiv:2509.02560*, 2025c.
- Shi, R., Chen, H., Zhang, Z., Liu, M., Xu, C., Wei, X., Chen, L., Zeng, C., and Su, H. Zero123++: a single image to consistent multi-view diffusion base model. *arXiv preprint arXiv:2310.15110*, 2023.
- Siddiqui, Y., Alliegro, A., Artemov, A., Tommasi, T., Sirigatti, D., Rosov, V., Dai, A., and Nießner, M. Meshgpt: Generating triangle meshes with decoder-only transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 19615–19625, 2024.
- Song, G., Zhao, Z., Weng, H., Zeng, J., Jia, R., and Gao, S. Mesh silksong: Auto-regressive mesh generation as weaving silk. *arXiv preprint arXiv:2507.02477*, 2025.

- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. Spectr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems*, 36:30222–30242, 2023.
- Tang, J., Ren, J., Zhou, H., Liu, Z., and Zeng, G. Dream-gaussian: Generative gaussian splatting for efficient 3d content creation. *arXiv preprint arXiv:2309.16653*, 2023.
- Tang, J., Chen, Z., Chen, X., Wang, T., Zeng, G., and Liu, Z. Lgm: Large multi-view gaussian model for high-resolution 3d content creation. In *European Conference on Computer Vision*, pp. 1–18. Springer, 2024a.
- Tang, J., Li, Z., Hao, Z., Liu, X., Zeng, G., Liu, M.-Y., and Zhang, Q. Edgerunner: Auto-regressive auto-encoder for artistic mesh generation. *arXiv preprint arXiv:2409.18114*, 2024b.
- Tao, C., Liu, Q., Dou, L., Muennighoff, N., Wan, Z., Luo, P., Lin, M., and Wong, N. Scaling laws with vocabulary: Larger models deserve larger vocabularies. *URL <https://arxiv.org/abs/2407.13623>*, 2024.
- Teng, Y., Shi, H., Liu, X., Ning, X., Dai, G., Wang, Y., Li, Z., and Liu, X. Accelerating auto-regressive text-to-image generation with training-free speculative jacobi decoding. *arXiv preprint arXiv:2410.01699*, 2024.
- Van Den Oord, A., Vinyals, O., et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, Y., Wang, J., and Qi, Y. We-gs: An in-the-wild efficient 3d gaussian representation for unconstrained photo collections, 2024. *URL <https://arxiv.org/abs/2406.02407>*.
- Wang, Y., Liu, H., Cheng, Z., Wu, R., Gu, Q., Wang, Y., and Wang, Y. Vocalnet: Speech llm with multi-token prediction for faster and high-quality generation. *arXiv preprint arXiv:2504.04060*, 2025.
- Wang, Y., Wang, J., and Qi, Y. Taking language embedded 3d gaussian splatting into the wild. *IEEE Transactions on Visualization and Computer Graphics*, 2026.
- Wang, Z., Lu, C., Wang, Y., Bao, F., Li, C., Su, H., and Zhu, J. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *Advances in Neural Information Processing Systems*, 36:8406–8441, 2023.
- Weng, H., Zhao, Z., Lei, B., Yang, X., Liu, J., Lai, Z., Chen, Z., Liu, Y., Jiang, J., Guo, C., et al. Scaling mesh generation via compressive tokenization. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 11093–11103, 2025.
- Wu, J., Zhang, C., Xue, T., Freeman, B., and Tenenbaum, J. Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling. *Advances in neural information processing systems*, 29, 2016.
- Wu, S., Lin, Y., Zhang, F., Zeng, Y., Xu, J., Torr, P., Cao, X., and Yao, Y. Direct3d: Scalable image-to-3d generation via 3d latent diffusion transformer. *arXiv preprint arXiv:2405.14832*, 2024.
- Xia, H., Ge, T., Wang, P., Chen, S.-Q., Wei, F., and Sui, Z. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. *arXiv preprint arXiv:2203.16487*, 2022.
- Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., and Yang, J. Structured 3d latents for scalable and versatile 3d generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 21469–21480, 2025.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pp. 38087–38099. PMLR, 2023.
- Xu, J., Cheng, W., Gao, Y., Wang, X., Gao, S., and Shan, Y. Instantmesh: Efficient 3d mesh generation from a single image with sparse-view large reconstruction models. *arXiv preprint arXiv:2404.07191*, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yi, T., Fang, J., Wang, J., Wu, G., Xie, L., Zhang, X., Liu, W., Tian, Q., and Wang, X. Gaussiandreamer: Fast generation from text to 3d gaussians by bridging 2d and 3d diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6796–6807, 2024.
- Zhang, L., Wang, Z., Zhang, Q., Qiu, Q., Pang, A., Jiang, H., Yang, W., Xu, L., and Yu, J. Clay: A controllable large-scale generative model for creating high-quality 3d assets. *ACM Transactions on Graphics (TOG)*, 43(4): 1–20, 2024.
- Zhao, R., Ye, J., Wang, Z., Liu, G., Chen, Y., Wang, Y., and Zhu, J. Deepmesh: Auto-regressive artist-mesh creation with reinforcement learning. *arXiv preprint arXiv:2503.15265*, 2025a.

Zhao, Z., Lai, Z., Lin, Q., Zhao, Y., Liu, H., Yang, S., Feng, Y., Yang, M., Zhang, S., Yang, X., et al. Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation. *arXiv preprint arXiv:2501.12202*, 2025b.

Zhu, J., Zhuang, P., and Koyejo, S. Hifa: High-fidelity text-to-3d generation with advanced diffusion guidance. *arXiv preprint arXiv:2305.18766*, 2023.

A. Implementation Details

Training was performed on two NVIDIA A800 GPUs and took approximately eight hours. We used AdamW as the optimizer ($\beta_1 = 0.9, \beta_2 = 0.99$). To improve training stability, we applied global norm clipping to the gradients, limiting their overall norm to within 1.0. The training procedure comprised two stages. During stage one, we trained only the decoding head, employing a cosine learning rate schedule decaying from 5×10^{-4} to 5×10^{-5} over 30 epochs. Subsequently, we applied LoRA to fine-tune the backbone, jointly training both modules for 10 epochs with a cosine learning rate schedule decaying from 1×10^{-4} to 1×10^{-5} . We set the LoRA rank to 16 and alpha to 32.

Test set composition. The composition of the test set is the same as in previous methods. We use image samples provided in the code repositories of Hunyuan3D 2.0 and Trellis as inputs, and generate 200 meshes as test examples to ensure that these data are unseen during training.

Input point clouds. The input point clouds are typically obtained either from scanned data or by sampling from the surfaces of meshes generated by diffusion-based models such as Trellis.

B. Test Dataset

Our test data was generated from the generation model (Xiang et al., 2025; Zhao et al., 2025b) and covers a rich and diverse set of shapes. Moreover, we categorized the shapes in the test dataset into three different difficulty levels: level-0, level-1, and level-2. (1) level-0: Simple shapes with minimal detail. (2) level-1: Relatively complex shapes with a certain amount of detail. (3) level-2: Challenging shapes featuring a rich array of details. In the entire test dataset, level-0 accounts for approximately 20%, level-1 for 40%, and level-2 for another 40%. We showcase a subset of the shapes from the test set in Figure 7.

C. Ablation of Sampling Strategies

We conducted a study of sampling strategies by comparing two methods: Independent Sampling (IS) and Top- K_s Probability Tree Sampling (PTS).

Independent Sampling. Samples are drawn independently from each decoding head’s probability distribution $p^{(d)}$, with token probabilities serving as sampling weights.

Top- K_s Probability Tree Sampling. For each layer’s distribution $p^{(d)}$, the Top- K_s tokens by probability are selected to recursively construct a probability tree. Denote the probabilities of the Top- K_s tokens at layer d by $\{m_{i,d,k}^{(d)}\}_{k=1}^{K_s}$. The weight of a path from the root to a leaf is then computed as $\prod_{d=1}^D m_{i,d,k}^{(d)}$. To constrain tree-construction complex-

Method	SCR $\uparrow$	Step Latency $\downarrow$	Speedup $\uparrow$
IS	2.021	47.83ms	$1.71 \times$
PTS($K_s = 2$)	2.030	48.06ms	$1.71 \times$
PTS($K_s = 3$)	2.033	48.47ms	$1.69 \times$
PTS($K_s = 4$)	2.036	49.08ms	$1.68 \times$

Table 4. Comparison of Independent Sampling (IS) and Top- K_s Probability Tree Sampling (PTS). Top- K_s PTS achieves a higher SCR, but due to the overhead of building the search tree at each iteration, its actual speedup is slightly lower than that of Independent Sampling.

Method	CD $\downarrow$	HD $\downarrow$	Avg. Lat. $\downarrow$
DeepMesh*	0.1323	0.2648	979.6s
BPT	0.1165	0.2223	257.6s
MeshSilk	0.1111	0.2293	294.8
MeshAnyV2	0.1708	0.3574	147.9s
TreeMeshGPT	0.1871	0.3425	239.3s
Ours	0.1168	0.2261	151.4s

Table 5. Broader comparison with other methods.

ity, branches with cumulative weights below 1×10^{-5} are pruned. Complete paths are then sampled according to their accumulated path probabilities.

Compared to IS, Top- K_s PTS improves the step compression ratio (SCR) by considering combinations among sampled tokens, but because each iteration requires building a search tree—incurring additional overhead—it does not achieve a higher speedup. The results are shown in Table 4.

D. User Study

We randomly selected 70 participants to complete a questionnaire as a subjective metric. Each questionnaire comprised 20 cases, resulting in 1,400 responses in total. Outputs from DeepMesh, BPT, and our method were randomly shuffled and anonymized to ensure fairness. For each case, participants were instructed to holistically evaluate both the generated shape and wireframe topology, then select the most favorable result. Owing to its tendency to generate fragmented and incomplete meshes, DeepMesh received relatively fewer votes. By contrast, participants struggled to distinguish between BPT and our method, resulting in nearly identical vote counts for these two approaches.

E. Analysis of Qualitative and Quantitative Comparisons

While DeepMesh can produce meshes with greater face counts and finer details, it requires substantially longer token sequences. To mitigate this, DeepMesh was trained with

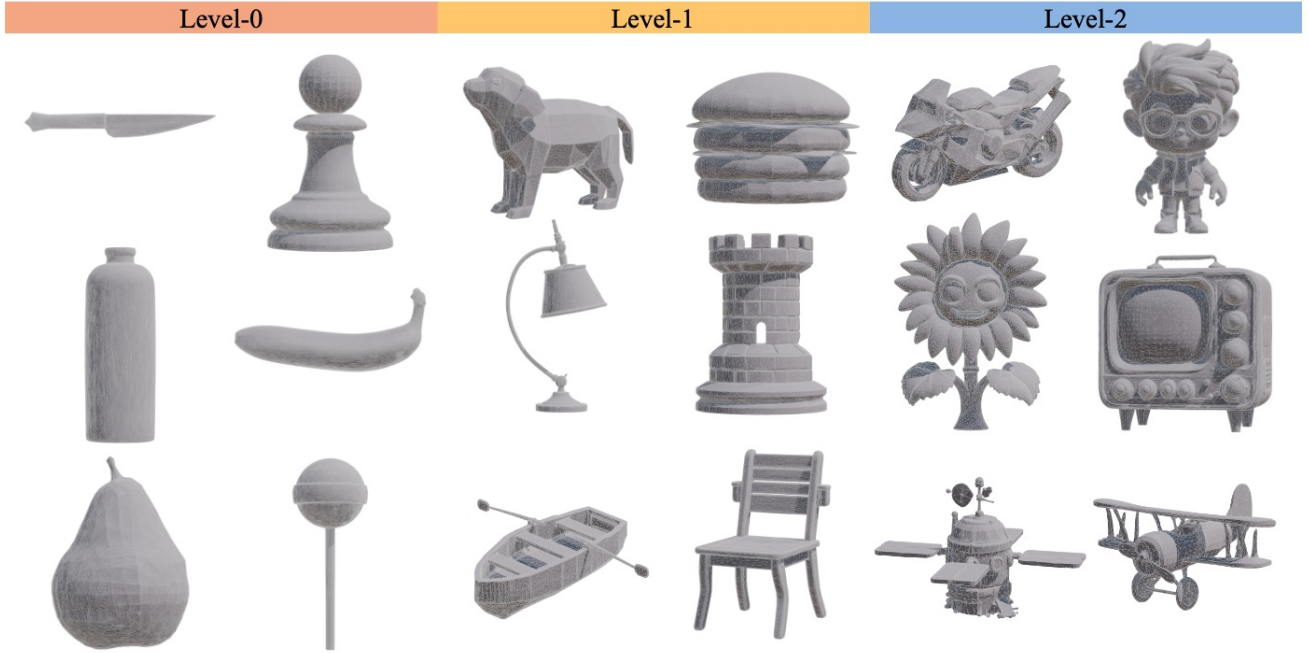


Figure 7. A subset of examples from the test dataset. Our test dataset contains a rich variety of shapes and is divided into three different difficulty levels: level-0, level-1, and level-2.

a truncated attention window and a maximum inference context size of 9,000 tokens—design decisions that result in fragmented meshes, as illustrated by the red boxes in Figure 4 of the main text. Furthermore, DeepMesh frequently produces meshes that are overly dense yet incomplete, as evidenced in rows 1 and 4. These shortcomings inflate its CD and HD metrics and diminish its user-study vote share.

In contrast, BPT omits any truncation window, resulting in more stable outputs and consistently robust performance across all test cases. The proposed XSpecMesh framework leverages BPT as its backbone: BPT’s token sequences are employed to train cross-attention decoding heads, and each candidate token generated by these heads are subsequently verified by the backbone model. This pipeline ensures that the generated outputs closely match those of BPT in terms of CD and HD, and—given the perceptual indistinguishability—yields a user-study vote share effectively equivalent to BPT’s. Finally, while preserving BPT-level quality, XSpecMesh achieves a $1.7\times$ speedup, thereby significantly reducing the backbone model’s inference time.

F. LoRA Instead of full Parameters Tuning

We fine-tune the backbone model using LoRA rather than full-parameter fine-tuning. Compared to full-parameter tuning, LoRA is more training-efficient and converges faster. Equally important, LoRA effectively prevents distribution drift in the backbone model. Since our method relies on the backbone to verify multiple candidate tokens, its predic-

tions are critical to generation quality. With full-parameter fine-tuning, gradients originating from the decoding heads can cause certain backbone parameters to drift significantly, harming sampling quality. By contrast, LoRA applies low-rank update matrices to the model; these low-rank updates curb any severe parameter drift induced by decoding-head gradients during training, thus preserving generation quality.

G. More Results

We further collected more examples, and displayed the generated results of BPT and our method in Figures 8 and 9. In these challenging cases, our approach is capable of producing meshes with shape and topology quality comparable to that of the base model BPT, while significantly accelerating the generation speed.

H. Broader Comparison

We provide a broader comparison with other autoregressive mesh generation methods in Table 5, including MeshSilk, MeshAnythingV2, and TreeMeshGPT. Among them, only MeshSilk achieves a better CD score than BPT, but its HD performance is worse than BPT and its generation latency is also higher. Therefore, considering both overall performance and inference efficiency, we believe BPT is currently the strongest method and is well suited as the backbone model for acceleration.

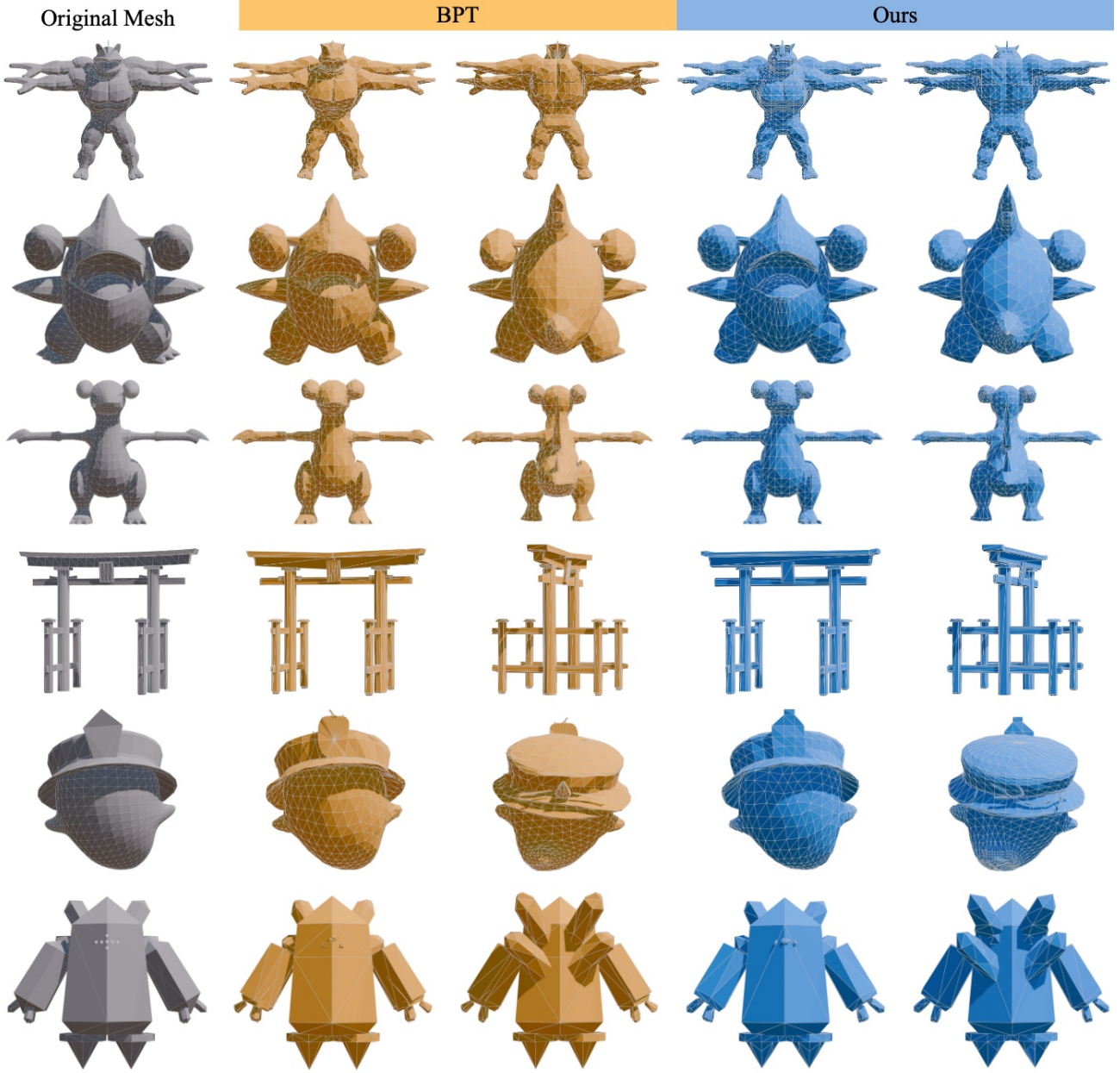


Figure 8. **Additional generation results of our method versus BPT.** Our acceleration method, built upon BPT, substantially accelerates generation while preserving BPT’s shape and topological fidelity.

I. Cross-Category Experiments

To further validate the generalization capability of our method, we conduct experiments on a cross-category dataset covering eight categories: monsters, humans, weapons, buildings, animals, plants, furniture, and decorative objects. The quantitative results are reported in Table 7. Our method performs well across categories, which can be attributed to the backbone model being extensively trained on a large-scale and diverse dataset, thereby endowing it with strong generalization ability that is not limited to specific cate-

gories.

J. Discussion

A concurrent work, FlashMesh(Shen et al., 2025a), also explores speculative decoding for accelerating mesh generation. Built on MeshXL’s uncompressed mesh encoding and an hourglass Transformer architecture, FlashMesh proposes a hierarchical speculative decoding strategy. However, its design is closely tied to the underlying mesh serialization scheme, making it difficult to generalize to irregular

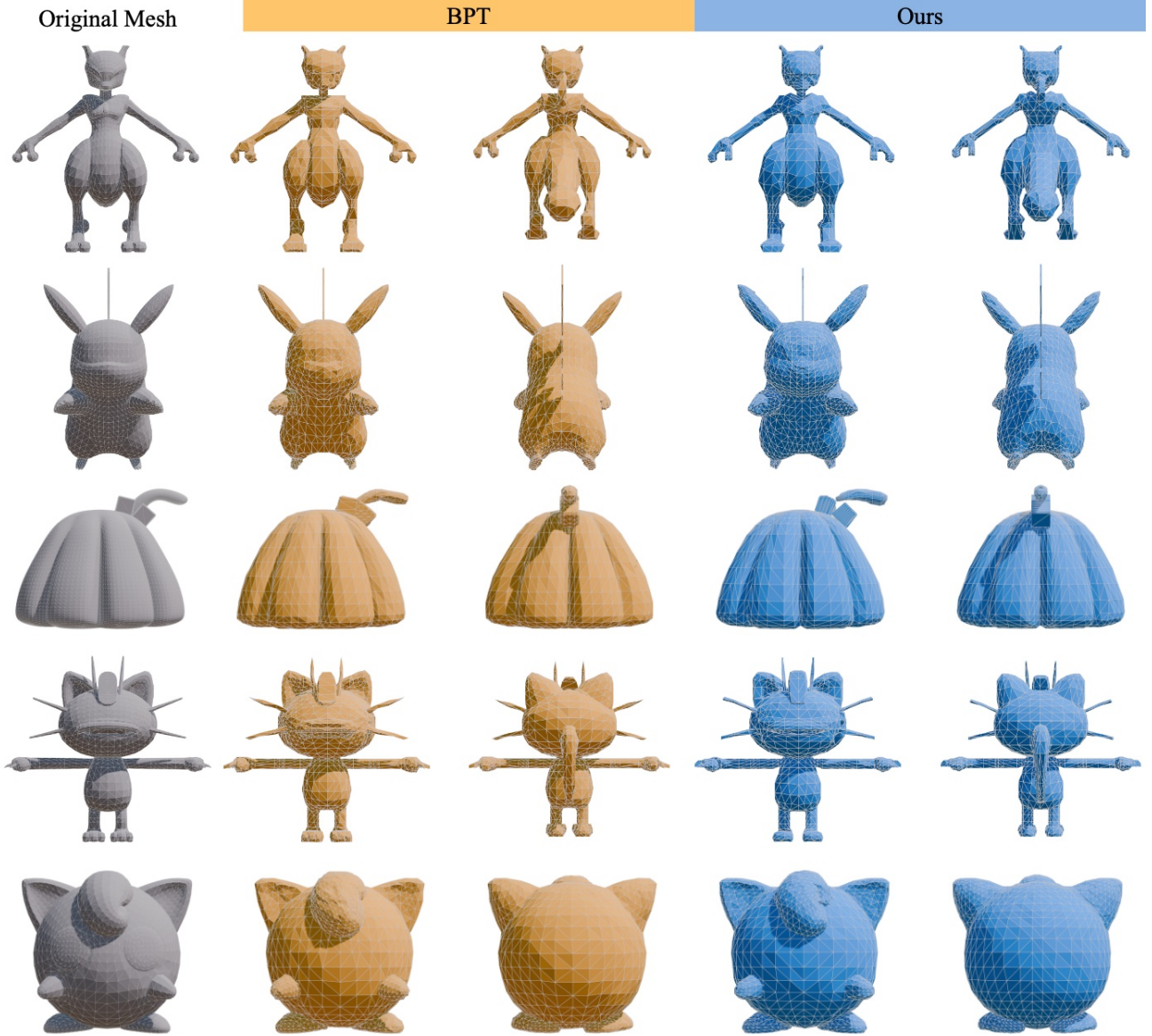


Figure 9. **Additional generation results of our method versus BPT.** Our acceleration method, built upon BPT, substantially accelerates generation while preserving BPT’s shape and topological fidelity.

compressed encodings such as those used in BPT and Mesh-Silk. In contrast, we propose a speculative decoding method that better adapts to irregular mesh serialization. By using cross-attention decoding heads to learn irregular serialization patterns, our method enables more general acceleration for mesh generation.

K. Generality

To further demonstrate the generality of our method, we also apply it to DeepMesh for acceleration. Using the same truncated-window training strategy as DeepMesh, the results are shown in Table 8.

Since DeepMesh adopts a hourglass transformer, its hierarchical structure introduces technical challenges for maintaining the KV cache. Specifically, the hourglass transformer performs hierarchical downsampling and upsampling over the sequence. We maintain a separate KV cache for each layer and record the valid KV cache positions (i.e., positions of tokens that have already been verified). Each time the KV cache is used, only the valid positions are accessed. When a token is verified, the valid positions in the KV cache are updated layer by layer. In addition, we employ hierarchical absolute position writes to the KV cache, meaning that at each forward pass, the newest values are written into the corresponding KV cache positions, overwriting previously

Method	Watertight Ratio $\uparrow$	Boundary Edge Ratio $\downarrow$	Two-Manifold Ratio $\uparrow$	Nonmanifold Edge Ratio $\downarrow$
DeepMesh	0.01	0.0569	0.01	0.0057
BPT	0.04	0.0362	0.04	0.0076
Ours	0.04	0.0320	0.04	0.0052
MeshSilk	0.00	0.0235	0.00	0.0002
MeshAnythingV2	0.00	0.1328	0.00	0.0052
TreeMeshGPT	0.08	0.0613	0.08	0.0061

Table 6. Evaluation metrics for watertightness and manifoldness.

Category	CD $\downarrow$	HD $\downarrow$
Monster	0.1176	0.2110
Human	0.1045	0.2232
Weapon	0.0904	0.2108
Building	0.1232	0.2459
Animal	0.0969	0.2097
Plant	0.1191	0.2302
Furniture	0.1087	0.2268
Decorative	0.0914	0.2195

Table 7. Category-wise CD and HD comparison.

Method	CD $\uparrow$	HD $\downarrow$	SCR $\uparrow$	Speedup $\uparrow$
MeshAnyV2	0.1708	0.3574	1.000	1.00×
+ours	0.1767	0.3639	1.975	1.63×
MeshSilk	0.1111	0.2293	1.000	1.00×
+ours	0.1182	0.2385	1.828	1.55×
DeepMesh	0.1323	0.2648	1.000	1.00×
+ours	0.1378	0.2767	2.184	1.52×
BPT	0.1165	0.2223	1.000	1.00×
+ours	0.1168	0.2261	2.021	1.71×

Table 8. The acceleration results of applying our method to different foundation models.

invalid entries.

L. Analysis of Watertightness and Manifoldness

Watertightness and manifoldness are important indicators of mesh quality and play a crucial role in downstream applications such as physical simulation, 3D printing, rendering, and geometry processing. Table 6 reports four topology-related metrics, including watertight ratio, boundary edge ratio, two-manifold ratio, and nonmanifold edge ratio, for different methods, including DeepMesh (Zhao et al., 2025a), BPT (Weng et al., 2025), MeshSilk (Song et al., 2025), MeshAnything v2 (Chen et al., 2024c), TreeMeshGPT (Lionar et al., 2025), and our method. In general, higher watertight and two-manifold ratios indicate better topological

soundness, while lower boundary edge and nonmanifold edge ratios indicate fewer structural defects in the generated meshes. The results show that autoregressive mesh generation remains challenging from a topological perspective. Most existing methods achieve relatively low watertight and two-manifold ratios, suggesting that guaranteeing strict mesh validity is still difficult at the current stage of autoregressive generation. At the same time, different methods exhibit different trade-offs: for example, TreeMeshGPT attains the highest watertight ratio and two-manifold ratio, while MeshSilk achieves the lowest boundary edge ratio and nonmanifold edge ratio. This indicates that current methods may be better at improving certain aspects of topology quality, but still struggle to consistently satisfy all desired mesh validity properties. Compared with the backbone model, our method maintains comparable performance on these topology-related metrics. Specifically, our method achieves the same watertight ratio and two-manifold ratio as BPT, while obtaining lower boundary edge ratio and nonmanifold edge ratio. Although our method does not achieve the best result on every individual metric, the overall results suggest that it preserves the topology quality of the backbone model while introducing inference acceleration. This observation further supports that our acceleration strategy does not significantly compromise mesh quality, even on challenging topological criteria such as watertightness and manifoldness.